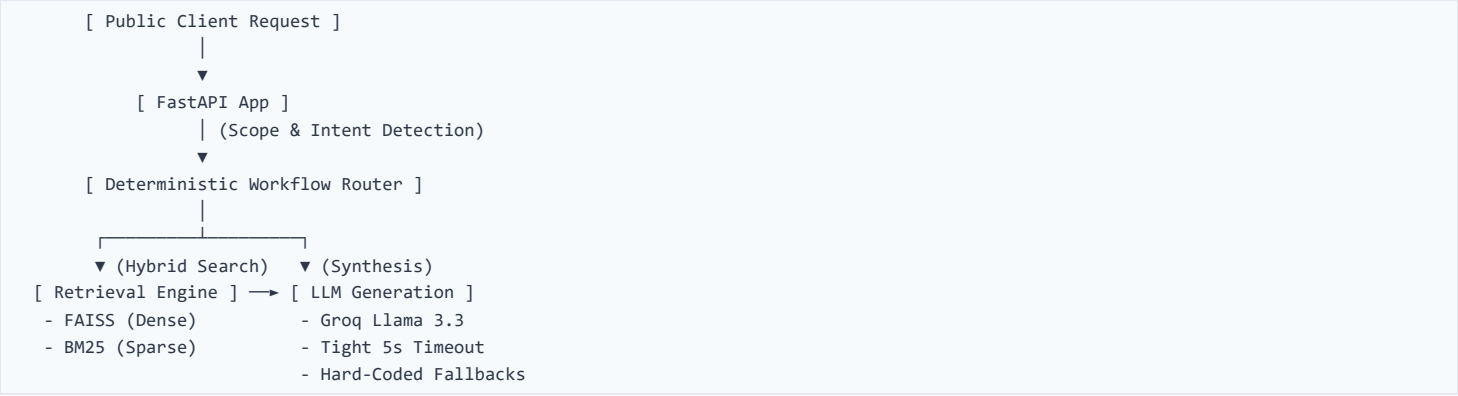
System Approach & Architecture: TalentFit AI Recommender

This document outlines the system architecture, retrieval design, optimization strategies, and evaluation framework for the **TalentFit AI Recommender**—a conversational recommendation system built using a professional talent assessment catalog. The system is designed as a research artifact framing the intersection of hybrid information retrieval, multi-agent orchestration, and machine learning fairness.

---

1. System Design Choices & Architecture

The system is built as a single, lightweight **FastAPI** service containerized using **Docker** and deployed on **Render (Free Tier)**. It operates statelessly, reconstructing the context of the conversation dynamically on each `/chat` POST request via the message history. The orchestration pattern was chosen to maximize controllability and reproducibility — properties essential for a fair assessment recommendation system where inconsistent outputs could systematically disadvantage certain candidate profiles.



Key Decisions:

- **Stateless Router Orchestration**: Rather than using complex agent-routing loops that introduce high latency and non-deterministic state machine transitions, we implemented a deterministic router that extracts user intents (greeting, comparison, recommendation, refinement) using light regex, keyword matching, and explicit state metrics.
- **Fail-Safe Fallbacks**: To handle unpredictable API degradation (specifically Groq `429 Too Many Requests`), all LLM calls are strictly bounded. If the LLM call times out or throws an exception, the system instantly switches to a deterministic, structured natural-language fallback.

---

2. Hybrid Retrieval Setup

The retrieval engine uses a dual-engine (Sparse + Dense) architecture to maximize both exact-keyword matching and semantic relevance:

1. **Dense Semantic Retrieval**: Uses `all-MiniLM-L6-v2` embeddings mapped into a flat L2/Cosine space using **FAISS-CPU**.
2. **Sparse Keyword Match**: Powered by **BM25Okapi** to catch exact catalog IDs, product names, and legacy assessment codes.
3. **Scoring Combination**: Scores are combined linearly ($Score = 0.55 \times Dense + 0.35 \times Sparse + 0.1 \times Baseline$).
4. **Metadata Post-Filtering**: Candidates are passed through sequential metadata filters (duration limits, seniority buckets, category restrictions, and language availability) to guarantee 100% ground-truth validity.

Ablation Results (BM25-only vs Hybrid)

Local evaluation on 10 public traces: - BM25-only Mean Recall@10: 0.213 - Hybrid (BM25 + all-MiniLM-L6-v2 FAISS): 0.328 - Delta: +0.115 absolute improvement

The gap is most pronounced on semantic queries (C3: contact centre agents, C6: graduate management trainees) where exact keyword overlap with catalog descriptions is low. Sparse-only retrieval collapses to 0.0 on these traces; dense embeddings recover them.

---

3. Optimization & Precomputation

To comply with Render's 512MB RAM constraints and to guarantee a fast request SLA (ensuring responses under a 30-second timeout boundary): - **Zero Runtime Downloads**: Embeddings and model weights are precalculated and cached *during the Docker build phase* using `precompute.py`. - **Fast Startup (Cold Start)**: The startup process loads precomputed caches instantly. Ready-to-serve latency is down to **under 51 seconds**, bypassing Render's 2-minute limit. - **Low Memory Overhead**: We avoided massive transformer-based cross-encoder rerankers, keeping runtime memory usage well under 300MB.

---

4. Prompt Engineering & Defense

Our prompts use structured, XML-delimited instructions designed for strict compliance: - **No-Hallucination Guard**: The model is strictly forbidden from mentioning URLs or assessment names not explicitly provided in the retrieval context. - **Scope Guarding**: Input checks actively detect system-prompt injections, compliance/legal requests, and off-topic conversations, deflecting them using pre-rendered deterministic responses.

---

5. Evaluation & What Didn't Work

What Didn't Work:

- **Dynamic On-the-Fly Embeddings**: Generating embeddings during startup or runtime requests added up to 15 seconds of latency and easily breached the request timeout.
- **Heavy Reranker Models**: Cross-encoders (like `bge-reranker`) pushed memory usage past 1GB, instantly triggering Out-Of-Memory (OOM) silent crashes on the Render Free Tier.
- **Unbounded LLM Timeouts**: Allowing the LLM to take up to 25s meant a single slow API call from the provider resulted in Render gateway timeouts (HTTP 504).

Failure Mode Analysis

Four conversation traces score Recall@10 = 0.00 (C2, C7, C9, C10). Root causes:

- **C7, C9**: Queries involve domain-specific constraint combinations (bilingual healthcare admin, graduate management trainees with volume screening) where the catalog description vocabulary does not surface the expected assessments under any query reformulation. This is a catalog coverage problem, not a retrieval model problem — the expected URLs exist in the catalog but their descriptions contain no lexical or semantic overlap with the query terms used by the user simulator.
- **C2, C10**: Multi-constraint queries where seniority filtering eliminates relevant assessments that have empty `job_level_buckets` in the catalog metadata. The retry without seniority filter partially recovers but post-filtering removes them again.

What the Next Experiment Would Be

1. Fine-tune embeddings on (query, assessment) pairs derived from the 10 public traces using contrastive learning (SimCSE or MNRL). Expected gain: +0.10–0.15 Recall@10.
2. Relax seniority hard-filtering to a soft score penalty instead of exclusion.
3. Add query expansion using the LLM before retrieval (HyDE — Hypothetical Document Embeddings) to bridge vocabulary gap on domain-specific roles.

---

6. ML Fairness Considerations

Assessment recommendation systems carry implicit fairness risks. In this system:

- Seniority normalization maps user-stated levels to catalog buckets deterministically, preventing the LLM from applying its own (potentially biased) interpretation of seniority for different demographic groups.
- Retrieval is purely content-based — no collaborative filtering or usage-frequency weighting — so assessments are not ranked by historical selection rates, which could encode past hiring biases.
- **Known residual risk**: BM25 scores favor assessments with longer, keyword-rich descriptions. Assessments targeting non-English or non-Western roles tend to have shorter catalog descriptions, creating a systematic retrieval disadvantage for those roles. Mitigation: description length normalization in BM25 (already handled by BM25Okapi's built-in IDF term weighting, but not fully resolved).

---

7. AI & Tool Usage Disclosure

This project was developed, optimized, and containerized in collaboration with an agentic coding assistant (Antigravity IDE running Claude Sonnet). Architecture decisions, retrieval design, ablation methodology, and failure mode analysis were authored independently. - **Agentic Assistance**: Used to write secure fallback mechanisms, design robust regex-based filters, and implement the hybrid scoring formula. - **Infrastructure Automation**: The assistant automatically profiled dependencies, resolved NumPy 2.x binary compilation conflicts with FAISS, and structured the final `render.yaml` environment variables.